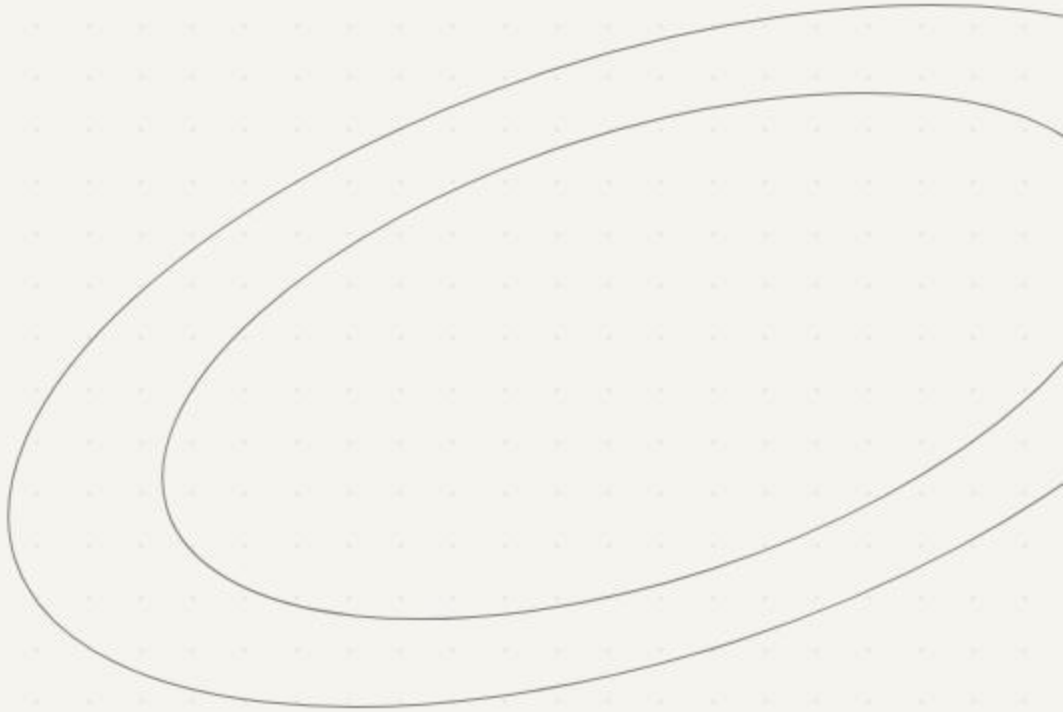
CELESTIAL MECHANICS × AI · 2026

From the next token to the next theorem

How a model works — and how I use it in practice



Bowen Liu

Overview: from token to theorem

Click a section card to jump; click a tag for a key page.

00 · OPENING + TOKEN

Token

Ten integers · vocabulary · billing

p. 3–8

Ten integers | Vocabulary

01 · MODEL

One function

Distribution · stateless

p. 9–10

Distribution | Stateless

02 · TRAINING & PROBLEM SOLVING

Learning from reward

RL loop · math and code · GRPO

p. 11–23

RL loop | GRPO formula | REFLEX

03 · SYSTEMS & ENGINEERING

Put the model in a system

Context · agents · code review

p. 24–40

Claude Code | System map

04 · AI × MATH

From tools to search

Acceleration · timeline · pattern

p. 41–46

Timeline | Pattern

05 · CONCLUSION

Back to practice

Workflow · compounding · close

p. 47–53

Two attempts | Close

6 sections · 52 pages · Click any card to start; `N` notes, `F` fullscreen.

RL: actions, outcomes and feedback

Act in an environment; use outcome feedback to improve action choices.

Observe environment → Take action → Outcome → Feedback / reward → Improve policy ↺

ENVIRONMENT

Produce an outcome

Actions affect the environment; new observations return.

FEEDBACK

Evaluate the result

Reward expresses progress or success.

POLICY

Improve action choices

Use experience to seek higher long-run return.

> RL uses **outcome feedback** to learn **how to choose actions**.

Reference: Sutton & Barto, Reinforcement Learning: An Introduction.

The GRPO objective

DeepSeekMath: group-relative advantage with clipping and a KL constraint.

$$J_{\mathrm{GRPO}}\left(\theta\right) = E_{q \sim P(Q),\, \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\mathrm{old}}}} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} L_{i,t} \right]$$

PER-TOKEN TERM

$$L_{i,t} = \min\left[\rho_{i,t}\left(\theta\right) \hat{A}_{i,t}, \mathrm{clip}\left(\rho_{i,t}\left(\theta\right), 1-\varepsilon, 1+\varepsilon \right) \hat{A}_{i,t} \right] - \beta D_{\mathrm{KL}}\left[\pi_\theta \| \pi_{\mathrm{ref}} \right]$$

PROBABILITY RATIO

$$\rho_{i,t}\left(\theta\right) = \frac{\pi_\theta\left(o_{i,t} \mid q, o_{i,<t}\right)}{\pi_{\theta_{\mathrm{old}}}\left(o_{i,t} \mid q, o_{i,<t}\right)}$$

GROUP-RELATIVE ADVANTAGE

$$\hat{A}_{i,t} = \frac{r_i - \mathrm{mean}\left(r_1, \ldots, r_G\right)}{\mathrm{std}\left(r_1, \ldots, r_G\right)}$$

Raise high-advantage responses while **clip and KL** limit policy drift.

Source: Shao et al., *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*, arXiv:2402.03300, 2024 · GRPO objective; implementation details vary.

GRPO: compare a group of answers

No critic; use relative performance within a group.

One question → Sample a group → Score each → Relative advantage ↻

ONE QUESTION · G = 4 ANSWERS

Reward / group advantage / update

A · r = 0.20	−1.5 ↓
B · r = 0.90	**+1.1 ↑**
C · r = 0.50	−0.4 ↓
D · r = 0.80	**+0.7 ↑**

Mean 0.60, std 0.27; illustrative values.

COMPETITION WITHIN A GROUP

Compare answers to one question, not absolute scores.

LEARN FROM BETTER OUTCOMES

Better answers gain probability; worse ones lose it.

NO CRITIC

Group mean is the baseline; no value model.

> Within one question, let **better answers win**.

REFLEX: search engine in the RL loop

E-commerce query expansion: generate → retrieve → reward result quality.

Query + top-k products → LLM expands → Search engine → Union of results → Reward ↺

REWARD · NO HUMAN LABELS

Relevance Rel(Y)/Rel(q) + coverage Ret(Y)/Ret(q); format violations score 0.

TRAINING

SFT warm start on 10k → GRPO on 100k real queries; 8×H200.

QUERIES WITH POSITIVE GAIN · %

SETTING	ΔCOV	ΔREL
SFT ONLY	12.8	8.9
GRPO ONLY	57.5	25.2
WARM START + GRPO	72.8	32.4

10,000 queries · 5 samples · LLaMA 3.2-3B

> GRPO turns **better retrieval** into an optimizable reward.

Zhang, Liu, et al., REFLEX, WSDM 2026 · ablation on this dataset and backbone.

Math and code share a feedback advantage

01 · CLOSED ENVIRONMENT

Axioms · Runtime

Math uses definitions, axioms and inference rules; code uses a language spec and runtime. The system is self-contained.

02 · TRACEABLE ERRORS

Locate the step

Proofs check inference step by step; tests, traces and line numbers locate code errors.

03 · CLEAN DATA

Parse and filter

Formal objects and code are structured, enabling bulk generation and verification.

Two levers: better policies + more compute for candidate generation and verification.

Closed environment + localizable errors + clean data → **cheap feedback, large-scale trial and error**.

Closure is relative: code still depends on inputs and libraries; modeling still needs formalization; data still needs curation.

Summary: environment, feedback, updates

Well-defined tasks let a model predict, receive checks and iterate in an environment.

Problem + environment → Next-token prediction → Check + trace → Define loss → Backpropagate ↺

01 · ENVIRONMENT

Clear rules & goal

The model can act, try and produce outputs.

02 · FEEDBACK

Evaluate & trace

Check outcomes and locate failing steps.

03 · UPDATE

Loss-driven iteration

Backpropagate, update parameters, improve.

> The environment supplies feedback; **loss turns it into parameter updates**.

When code outruns human review

Claude Code's review problem: code scales faster than human attention.

SCALE PRESSURE · ANTHROPIC MONOREPO

2,500 → 30,000+

Changes landing per day · May → Sep 2026

Claude Code has a separate repository.

PRODUCTION

Agents write more code, faster.

BOTTLENECK

Human review time stays limited.

CONSEQUENCE

Line-by-line review cannot keep up.

Goal: reduce **review + repair time**, with reverts and incidents as guardrails.

> The next gain comes from **redesigning review**.

Source: Osmani, Lunt & Wang · 15 Oct 2026 · How the Claude Code team does code review · Team-reported practice.

From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

Probability gives breadth

Verification gives correctness

People set the direction

> The next theorem still begins with the **next token**.